

深圳大学实验报告

课程名称： 编译原理

实验项目名称： 高级语言及其文法

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 王祎乐

报告人：__ 学号：__ 班级： __

实验时间： 2026年4月11日

实验报告提交时间： 2026年4月15日

教务处制

实验目的与要求：

目的：通过实验，加深对文法及其分类的理解。

要求：

第一部分：文法的分类

输入：读入一个名为“test*.in”，表示一个文法的文本文件。文件由 n 行组成，每行表示一个产生式，产生式是形如“ $S \rightarrow a$ ”的字符串，终结符由小写字母，非终结符由大写字母表示，第一行的第一个非终结符为开始符号。共有十个测试用例（ps：可能包含空串 ϵ ）。

处理：对文法进行分类

输出：控制台输出，明确这是一个 0/1/2/3 型文法

基本框架已写好，实现函数即可

方法、步骤：

要完成本实验，依据实验要求进行分解，需要完成的实验步骤是：

1. 0, 1, 2, 3 型文法之间有什么关系？

每种文法的定义都基于其产生式规则的形式，随着文法类型编号的增加，规则的限制越来越严格。

关系：

(1) 分类层次：这四种文法呈现出一个逐层包含的结构，即 3 型文法是 2 型文法的子集，2 型文法是 1 型文法的子集，1 型文法又是 0 型文法的子集。用数学集合的概念来表示就是： $3 \text{ 型文法} \subseteq 2 \text{ 型文法} \subseteq 1 \text{ 型文法} \subseteq 0 \text{ 型文法}$ 。

(2) 规则限制程度：每种文法的定义都基于其产生式规则的形式，随着文法类型编号的增加，规则的限制越来越严格。

① 0 型文法（短语文法）：这是最宽泛的文法类型，对产生式 $\alpha \rightarrow \beta$ 没有任何限制，只要 α 至少包含一个非终结符即可。

② 1 型文法（上下文有关文法）：相较于 0 型文法，它增加了对产生式长度的限制，对于产生式 $\alpha \rightarrow \beta$ ，有 $|\alpha| \leq |\beta|$ ，除非 β 是 ϵ 。同时，若 $S \rightarrow \epsilon$ 是产生式，则 S 不能出现在任何产生式的右部。

③ 2 型文法（上下文无关文法）：进一步限制了产生式的左部必须是单个非终结符，产生式的形式为 $A \rightarrow \beta$ ，其中 A 是单个非终结符， β 是任意的终结符和非终结符的串。

④ 3 型文法（正则文法）：是限制最严格的文法类型，分为左线性文法和右线性文法。右线性文法的产生式形式为 $A \rightarrow aB$ 或 $A \rightarrow a$ ，左线性文法的产生式形式为 $A \rightarrow Ba$ 或 $A \rightarrow a$ ，其中 A, B 是非终结符， a 是终结符

2. 如何判定一个文法是 0、1、2、3 型文法？

思路：

(1) 判断是否为 3 型文法：遍历所有产生式，检查是否符合右线性或左线性文法的形式。

①初始化标志变量

- 初始化两个布尔变量 `is_right_linear` 和 `is_left_linear` 为 `true`，分别表示文法是否为右线性文法和左线性文法

②遍历所有规则：

- 函数遍历文法中的所有产生式规则 `rules`。
- 对于每条规则，使用 `find(' ')` 方法找到规则中左右部分的分隔位置（空格的位置）。
- 通过 `substr()` 方法分别提取规则的左部 `left` 和右部 `right`。

③左边部分的检查：

- 检查规则左部的长度是否为 1，且左部的字符是否为非终结符。这是因为 3 型文法要求产生式左部必须是单个非终结符。
- 如果左部不满足这个条件，说明该规则不符合 3 型文法的要求，函数直接返回 `false`。

④右边部分的检查：

- 右部长度为 1 的情况：如果右部长度为 1，检查该字符是否为终结符。若不是终结符，则不符合 3 型文法规则，函数返回 `false`。
- 右部长度为 2 的情况：
 - 如果右部第一个字符是终结符，第二个字符是非终结符，说明该规则不符合右线性文法的形式，将 `is_right_linear` 置为 `false`。
 - 如果右部第一个字符是非终结符，第二个字符是终结符，说明该规则不符合左线性文法的形式，将 `is_left_linear` 置为 `false`。
 - 如果右部的字符组合既不是上述两种情况之一，说明该规则不符合 3 型文法要求，函数返回 `false`。
- 右部长度不为 1 或 2 的情况：如果右部长度不是 1 也不是 2，说明该规则不符合 3 型文法的形式，函数返回 `false`。

⑤返回结果：

- 遍历完所有规则后，只要文法符合左线性文法或者右线性文法其中之一，函

数就返回 `true`，表示该文法是 3 型文法；否则返回 `false`。

(2) 判断是否为 2 型文法：检查每个产生式的左部是否为单个非终结符。2 型文法（上下文无关文法）：产生式的形式为 $A \rightarrow \beta$ ，其中 A 是单个非终结符， β 是任意的终结符和非终结符的串。

①遍历所有规则：

- 函数遍历文法中的所有规则，检查每条规则的左边部分是否符合类型 2 文法的要求。

②左边部分的检查：

- 对于每条规则，函数首先找到左边和右边的分隔位置（通过空格 `' '` 分隔）。
- 提取左边部分 `left`，并检查其长度是否为 1，且是否为非终结符。
 - 如果左边部分的长度不为 1，或者左边部分的字符不是非终结符，函数直接返回 `false`，表示该文法不是类型 2 文法。

③返回结果：

- 如果所有规则的左边部分都满足类型 2 文法的要求，函数返回 `true`，表示该文法是类型 2 文法。

(3) 判断是否为 1 型文法：检查每个产生式的左部长度是否小于等于右部长度，同时处理空串产生式的特殊情况。

①起始符号和空串检查：

- 首先，函数获取文法的起始符号 `start_symbol`，即第一条规则的左边部分。
- 然后，函数遍历所有规则，检查是否存在空串（ ϵ ）的产生式。如果存在空串，则必须满足以下条件：
 - 空串的产生式左边必须是起始符号，并且长度必须为 1。
 - 如果空串的产生式不满足上述条件，函数直接返回 `false`，表示该文法不是类型 1 文法。

②长度检查：

- 对于非空串的产生式，函数检查左边符号串的长度是否小于或等于右边符号串的长度。如果左边长度大于右边长度，函数返回 `false`，表示该文法不是类

型 1 文法。

③起始符号的额外检查：

- 如果文法中存在空串的产生式（即 `has_start_epsilon` 为 `true`），函数会进一步检查起始符号是否出现在任何规则的右边部分。如果起始符号出现在任何规则的右边，函数返回 `false`，因为类型 1 文法不允许起始符号出现在任何规则的右边（除非是空串的产生式）。

④返回结果：

- 如果所有检查都通过，函数返回 `true`，表示该文法是类型 1 文法。

（4）判断是否为 0 型文法：按照 3 型、2 型、1 型的顺序依次判断，如果都不满足则为 0 型文法

实验过程及内容：

判断 3 型文法：

```

25 // 检查是否所有产生式符合3型文法且方向一致
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
26 bool is_type_3() {
27     // 初始化左右线性标志
28     bool is_right_linear = true;
29     bool is_left_linear = true;
30
31     for (const string &rule : rules) {
32         // 找到左右部分隔的空格
33         size_t space_pos = rule.find(' ');
34         if (space_pos == string::npos) {
35             return false; // 格式错误, 无分隔符
36         }
37         string left = rule.substr(0, space_pos);
38         string right = rule.substr(space_pos + 1);
39
40         // 检查左部: 必须是单个非终结符
41         if (left.length() != 1 || !is_non_terminal(left[0])) {
42             return false;
43         }
44
45         // 检查右部长度
46         int right_len = right.length();
47         if (right_len == 1) {
48             // 右部长度1: 必须是终结符
49             if (!is_terminal(right[0])) {
50                 return false;
51             }
52         } else if (right_len == 2) {
53             // 右部长度2: 检查左右线性
54             char c1 = right[0];
55             char c2 = right[1];
56             bool is_c1_term = is_terminal(c1);
57             bool is_c1_nonterm = is_non_terminal(c1);
58             bool is_c2_term = is_terminal(c2);
59             bool is_c2_nonterm = is_non_terminal(c2);
60
61             // 右线性要求: 终结符 + 非终结符 → 不满足则右线性为false
62             if (!(is_c1_term && is_c2_nonterm)) {
63                 is_right_linear = false;
64             }
65             // 左线性要求: 非终结符 + 终结符 → 不满足则左线性为false
66             if (!(is_c1_nonterm && is_c2_term)) {
67                 is_left_linear = false;
68             }
69             // 既不是右线性也不是左线性的组合, 直接返回false
70             if (!is_c1_term && !is_c1_nonterm) return false;
71             if (!is_c2_term && !is_c2_nonterm) return false;
72             if ((is_c1_term && is_c2_term) || (is_c1_nonterm && is_c2_nonterm)) {
73                 return false;
74             }
75         } else {
76             // 右部长度不是1或2, 不符合3型文法
77             return false;
78         }
79     }
80
81     // 只要满足左线性或右线性其一即为3型
82     return is_right_linear || is_left_linear;
83 }
84

```

判断 2 型文法:

```
85 // 2型文法: 左部必须是单个非终结符
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
86 bool is_type_2() {
87     for (const string &rule : rules) {
88         size_t space_pos = rule.find(' ');
89         if (space_pos == string::npos) {
90             return false; // 格式错误
91         }
92         string left = rule.substr(0, space_pos);
93         // 检查左部: 单个非终结符
94         if (left.length() != 1 || !is_non_terminal(left[0])) {
95             return false;
96         }
97     }
98     // 所有规则左部都满足则为2型
99     return true;
100 }
```

判断 1 型文法:

```
102 // 1型文法: 左部长度 ≤ 右部长度, 且左部至少含一个非终结符, 允许 A → ε
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
103 bool is_type_1() {
104     if (rules.empty()) return false;
105
106     // 获取起始符号 (第一条规则的左部)
107     size_t first_space = rules[0].find(' ');
108     if (first_space == string::npos) return false;
109     string start_symbol = rules[0].substr(0, first_space);
110     bool has_start_epsilon = false;
111
112     // 第一步: 检查空串产生式
113     for (const string &rule : rules) {
114         size_t space_pos = rule.find(' ');
115         if (space_pos == string::npos) return false;
116         string left = rule.substr(0, space_pos);
117         string right = rule.substr(space_pos + 1);
118
119         // 检查空串ε产生式
120         if (right == "ε") {
121             // 空串产生式的左部必须是起始符号且长度为1
122             if (left != start_symbol || left.length() != 1) {
123                 return false;
124             }
125             has_start_epsilon = true;
126         }
127     }
128
129     // 第二步: 检查所有产生式的长度和非终结符要求
130     for (const string &rule : rules) {
131         size_t space_pos = rule.find(' ');
132         string left = rule.substr(0, space_pos);
133         string right = rule.substr(space_pos + 1);
134
135         // 非空串产生式: 左部长度 ≤ 右部长度
136         if (right != "ε" && left.length() > right.length()) {
137             return false;
138         }
139
140         // 检查左部至少包含一个非终结符
141         bool has_non_terminal = false;
142         for (char c : left) {
143             if (is_non_terminal(c)) {
144                 has_non_terminal = true;
145                 break;
146             }
147         }
148         if (!has_non_terminal) {
149             return false;
150         }
151     }
152
153     // 第三步: 如果有起始符号的空串产生式, 检查起始符号是否出现在任何规则的右部
154     if (has_start_epsilon) {
155         for (const string &rule : rules) {
156             size_t space_pos = rule.find(' ');
157             string right = rule.substr(space_pos + 1);
158             // 起始符号出现在右部 → 不符合1型
159             if (right.find(start_symbol) != string::npos) {
160                 return false;
161             }
162         }
163     }
164
165     return true;
166 }
```

判断 0 型文法:

```
168 int classify_grammar() {
169     if (is_type_3()) return 3;
170     if (is_type_2()) return 2;
171     if (is_type_1()) return 1;
172     return 0;
173 }
174
```

实验结论:

1. 文法分类的实验结果

```
● → /workspace git:(master) X g++ -std=c++11 -O2 -o ex1 program.cpp
./ex1
case0: 2
case1: 3
case2: 2
case3: 2
case4: 0
case5: 1
case6: 3
case7: 2
case8: 3
case9: 1
○ → /workspace git:(master) X
```

该结果表明程序成功对每个测试用例的文法进行了分类，输出了它们所属的文法类型

心得体会:

本次实验通过对文法分类的实现，我深刻理解了乔姆斯基文法体系的核心思想，并掌握了不同类型文法的判定方法。

文法分类:

乔姆斯基的四类文法（0-3 型）是一个规则限制逐渐严格的体系：

- (1) **0 型文法**（短语文法）：无规则限制，表达能力最强，对应图灵机。
- (2) **1 型文法**（上下文有关文法）：要求产生式左部长度 $\leq$ 右部长度（除空串），对应线性有界自动机。
- (3) **2 型文法**（上下文无关文法）：左部必须为单个非终结符，对应下推自动机，适用于编程语言的语法描述（如表达式、语句结构）。
- (4) **3 型文法**（正则文法）：右部只能是“终结符 + 非终结符”或“终结符”，对应有限自动机，适用于词法分析（如标识符、关键字识别）。

文法推导：

文法推导是从开始符号出发，通过反复应用产生式规则生成句子的过程。例如，对于文法 $S \rightarrow aSb \mid \epsilon$ （上下文无关文法），其推导过程为：

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

该过程体现了上下文无关文法的“递归嵌套”特性。而正则文法的推导则呈现线性结构，例如 $S \rightarrow aA \mid b$ ，其推导只能是：

$$S \Rightarrow aA \Rightarrow aaA \Rightarrow aab \quad \text{或者是} \quad S \Rightarrow b$$

指导教师批阅意见：

成绩评定：

指导教师签字：
年 月 日

备注：